FIT3155: Week 9 Lab Sheet

(Questions are based on Week 8 topic dealing with Amortized analysis)

Objectives: Develop an understanding of amortized complexity, including common analysis techniques: aggregate, accounting, and potential methods.

Conceptual exercises

☑ 1. Referring back to the k-bit binary counter, analyze the amortized complexity using:

- ☑ (a) Aggregate method
- ☑ (b) Accounting method
- ☑ (c) Potential method

☑ 2. For a Fibonacci heap, using the potential function

$$\text{nTrees}(H) + 2\text{nMarked}(H),$$

characterize the amortized complexities of these operations:

- ☑ (a) insert
- ☑ (b) extract-min
- ☑ (c) decrease-key

☐ 3. **Background** A dynamic array is a commonly used data structure that stores a collection of elements contiguously in memory while allowing its size to vary during execution, and still providing random access to its elements. (In practice, a Python `list` is implemented as a dynamic array.)

A dynamic array begins with an initial fixed capacity. As elements are appended, the array may eventually become full. When this happens, a new, larger block of memory is allocated (typically with double the previous capacity), and all existing elements are copied into this new block before the old memory is released. The new element is then appended into the resized array. Although this resizing step is costly, it occurs infrequently.

For this question, assume that the dynamic array starts off empty. The first `append` initializes an array of capacity 1 into which the element is inserted. Subsequently, whenever the array becomes full, its capacity is doubled. This defines the following sequence of resizes:

$$0 \to (1 = 2^0) \to (2 = 2^1) \to (4 = 2^2) \to (8 = 2^3) \to (16 = 2^4) \to \cdots .$$

Question: Consider a sequence of n `append` operations, where each `append` pushes an element to the end of the dynamic array. Your task is to analyze the amortized cost of the `append` operation using the three methods of amortized analysis covered in this unit. Specifically:

- ☐ (a) Using the **aggregate method**, determine the amortized cost of each `append` operation.
- ☐ (b) Using the **accounting method**, analyze the amortized cost of each `append` operation.
- ☐ (c) Using the **potential method**, analyze the amortized cost of each `append` operation.
    - Use the following potential function, defined as a linear combination of the array size (denoted by $\text{size}(A_i)$) and the array capacity (denoted by $\text{capacity}(A_i)$) in any current state A_i of the dynamic array:

$$\Phi(A_i) = 2 \times \text{size}(A_i) - \text{capacity}(A_i).$$

        - **Note:** Since the array always doubles when an append is attempted on a full array (when $\text{size}(A_i) = \text{capacity}(A_i)$), then in every state A_i, we have

$$\text{size}(A_i) \geq \frac{1}{2}\text{capacity}(A_i).$$

          This ensures that $\Phi(A_0) = 0$ and $\Phi(A_i) \geq 0$ for all $i > 0$. This property guarantees that the sum of amortized costs over any sequence of n `append` operations upper-bounds the sum of the actual costs over that sequence.

        - Solutions to this question will motivate how the above potential function could be arrived at, starting with a general linear combination of terms $\Phi(A_i) = \lambda_1 \text{size}(A_i) + \lambda_2 \text{capacity}(A_i)$, although the process of deriving potential functions is non-examinable in this unit.